# Ensemble Learning

**Fateha Khanam Bappee, Ph.D.**
Department of Computer Science and Telecommunications Engineering
Noakhali Science and Technology University
March 2025

Modified from the slides by

Dr. Amilcar Soares, Dr. Raymond J. Mooney, Dr. Pascal Poupart

# Outline

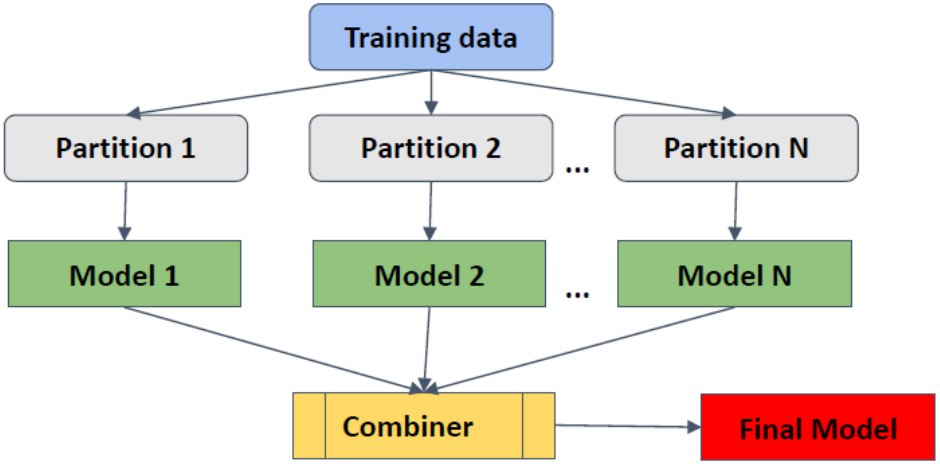
- Ensemble Learning in General
- Two most popular ensemble methods:
  - Bagging
    - Random Forest
  - Boosting
    - AdaBoost
    - Gradient Boosting
    - XGBoost

# Ensemble Learning in General

- Previous methods
  - Learn a single hypothesis, chosen form a hypothesis space that is used to make predictions.
- Which technique should we pick?
- Ensemble learning
  - select a collection (ensemble) of hypotheses and combine their predictions.
- Key motivation
  - WISDOM OF THE CROWD. Reduce the error rate. It will become much more unlikely that the ensemble will misclassify an example.

# Ensemble Learning

- Learn multiple alternative models using different training data or different learning algorithms.
- Combine decisions of multiple models.
- Less bias and less variance

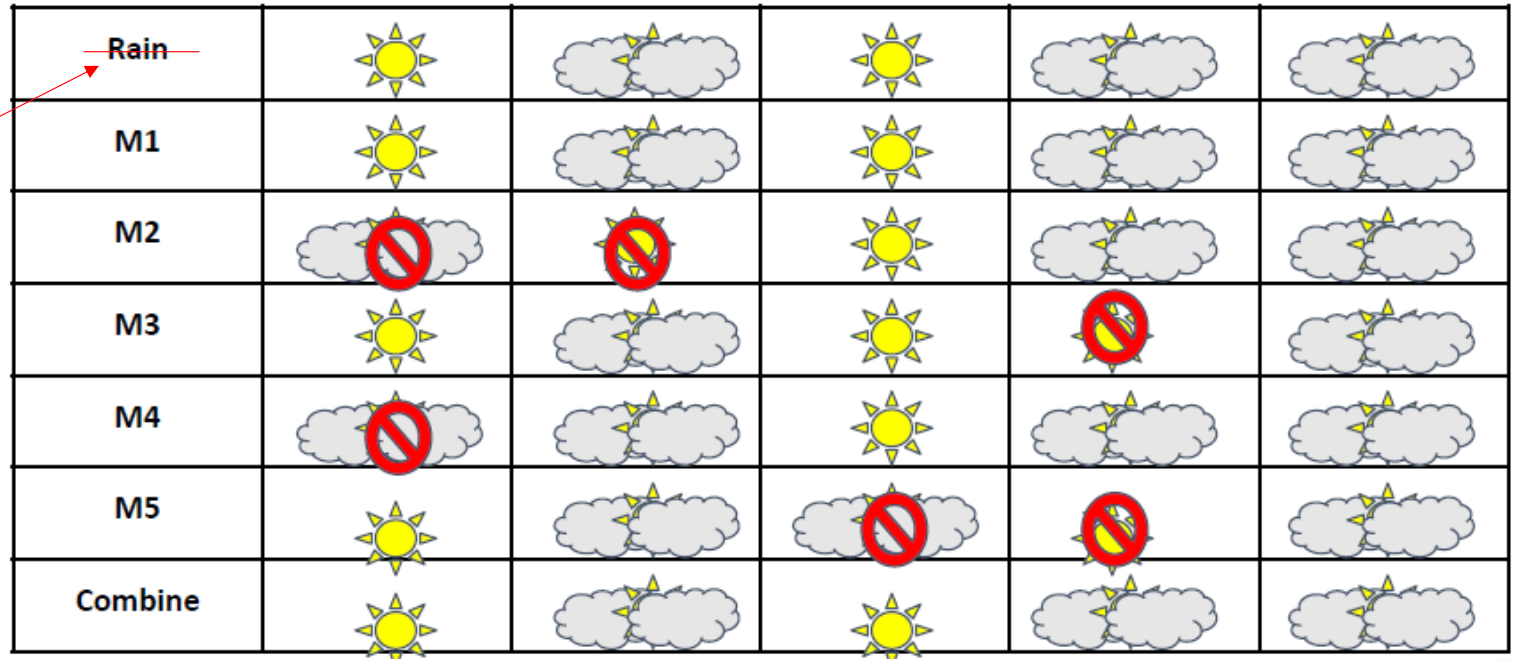


# Ensemble Learning

- No Free Lunch theorem [1]:
  - There is no algorithm that is always the most accurate.
- The value of ensembles
  - When combining multiple independent and diverse decisions, each of which is at least more accurate than random guessing, random errors cancel each other out, correct decisions are reinforced.

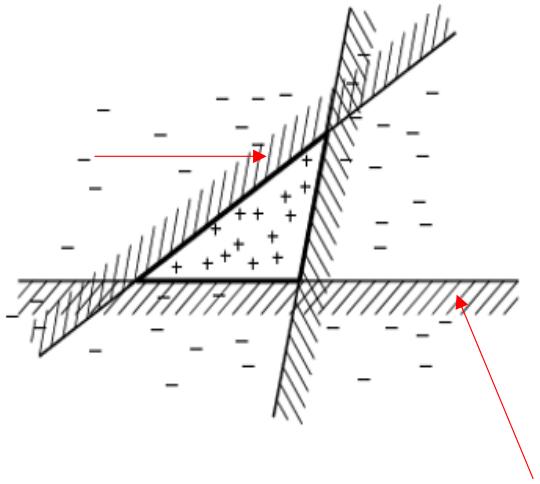
[1] Wolpert, D.H., Macready, W.G. (1997), "No Free Lunch Theorems for Optimization", *IEEE Transactions on Evolutionary Computation* 1, 67.

# Ensemble Learning (Example)



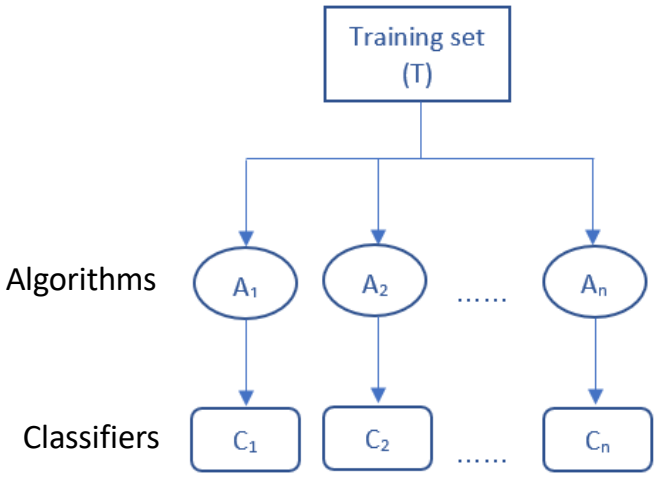
# Ensemble Learning (Example)

- Enlarging the hypothesis space
  - Perceptrons (linear separators)
  - Ensemble of perceptrons

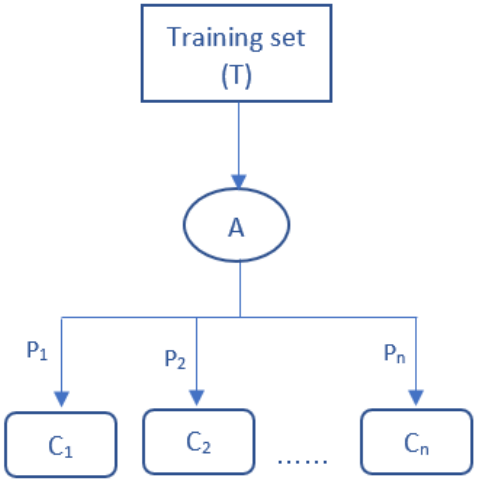


- Three linear threshold hypothesis (positive examples on the non-shaded side);
- Ensemble classifies as positive any example classified positively be all three.
- The resulting triangular region hypothesis is not expressible in the original hypothesis space.

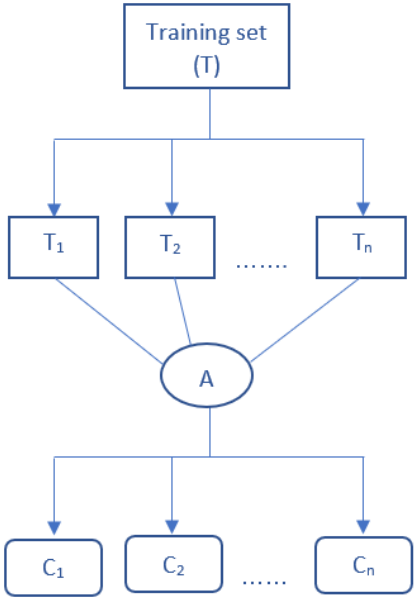
# Different Types of Ensemble Learning




Different algorithms, same set of training data



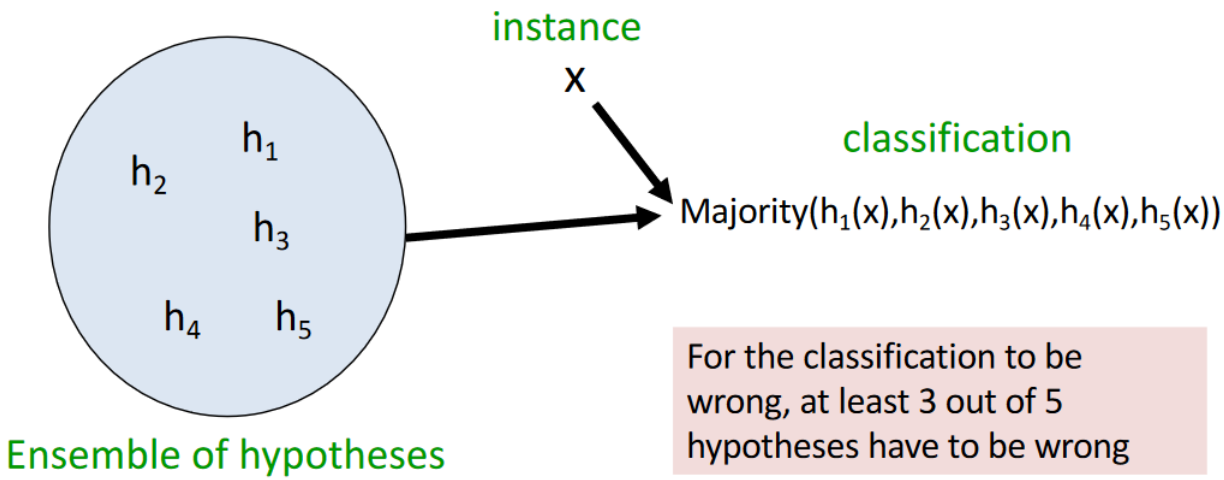

Same algorithm, different parameter settings




Same algorithm, different subsets of data.
Example: Bagging, Boosting

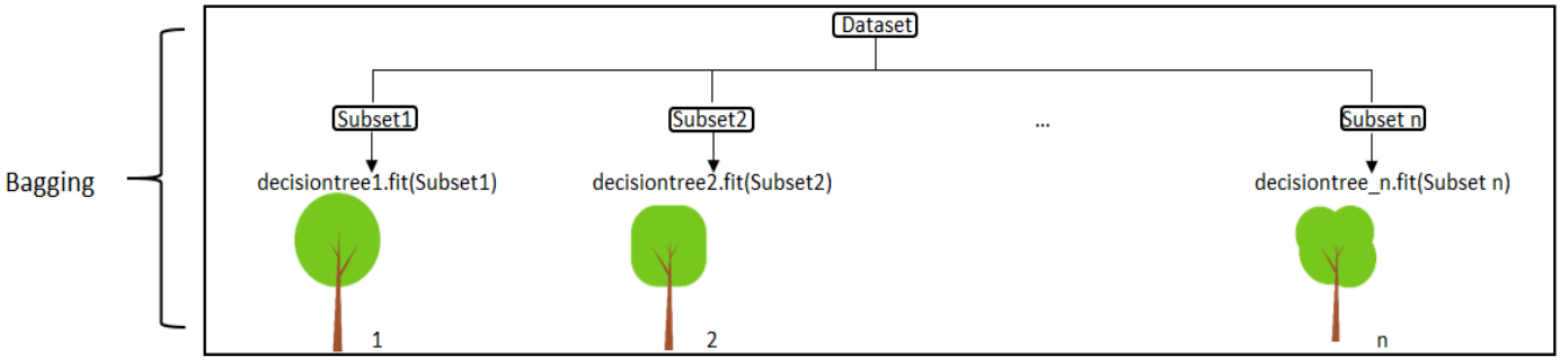
# Bagging

- Bootstrap Aggregation
- Majority voting



# Bagging

- Training a bunch of individual models parallelly. Each model uses a random subset of data for training




https://towardsdatascience.com/basic-ensemble-learning-random-forest-adaboost-gradient-boosting-step-by-step-explained-95d49d1e2725

# Bagging

- Bootstrapping Process

# Random Forest

- This algorithm was first introduced by Tin Kam Ho and then an extension of it was developed by Leo Breiman (2001)
- Definition:
  - Random forest is an ensemble classifier that consists of many decision trees and outputs the class that is the most voted of the class's output by individual trees.
  - Generally, CART trees are used with this method.
- Motivation: reduce error correlation between classifiers
- Main idea: build a larger number of un-pruned decision trees
- Key: using a random selection of features to split  on at each node

# How Random Forest Works?

Step 1: Select n random subsets from the training set

Step 2: Train n decision trees

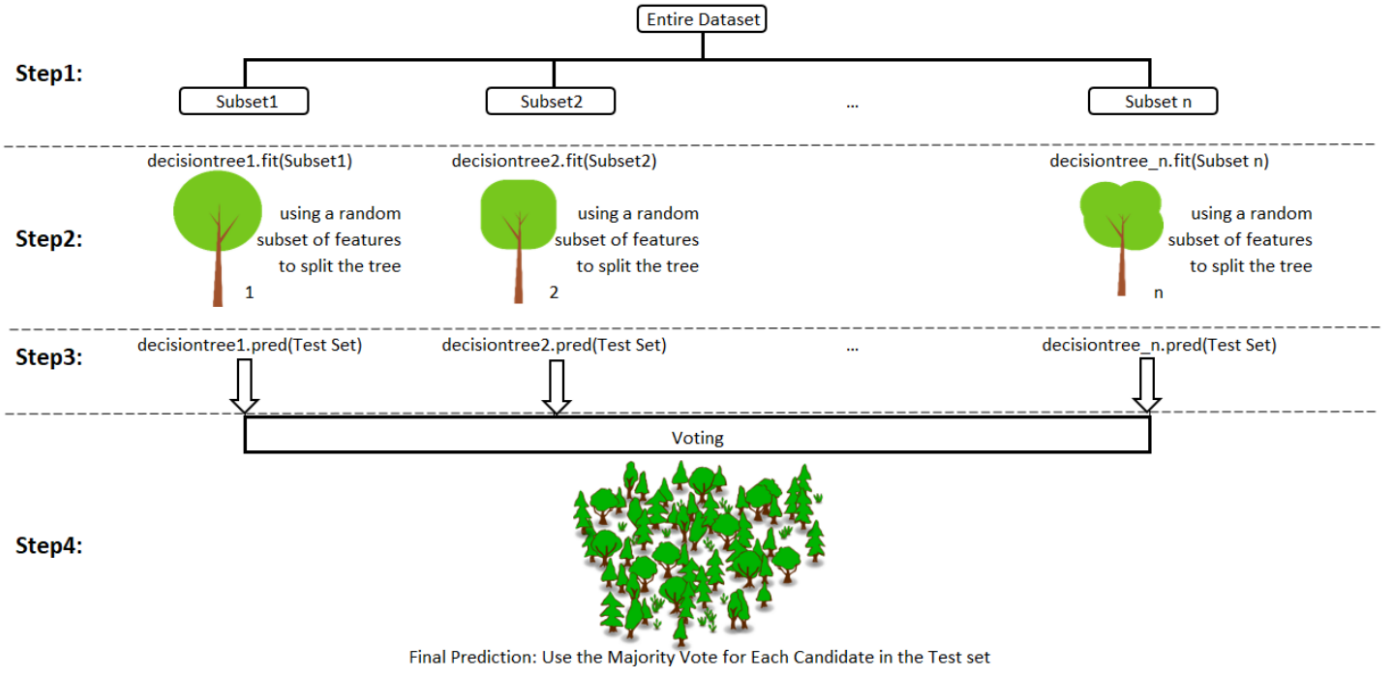
- one random subset is used to train one decision tree
- the optimal splits for each decision tree are based on a random subset of features

Step 3: Each individual tree predicts the records/candidates in the test set, independently.

Step 4: Make the final prediction

- For each candidate in the test set, Random Forest uses the class with the majority vote (classification) as this candidate's final prediction.

# Random Forest Example




https://towardsdatascience.com/basic-ensemble-learning-random-forest-adaboost-gradient-boosting-step-by-step-explained-95d49d1e2725

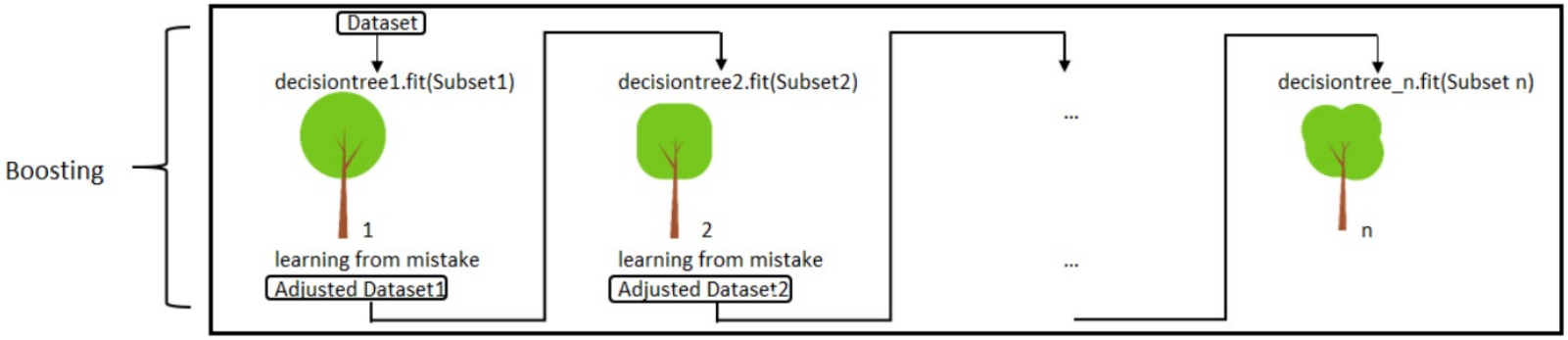
# Random Forest

- **Advantages**
  - It is one of the most accurate learning algorithms available. For many data sets, it produces a highly accurate classifier.
  - It runs efficiently on large databases.
  - It can handle thousands of input variables without variable deletion.
  - It generates an internal unbiased estimate of the generalization error as the forest building progresses.
  - It has an effective method for estimating missing data and maintains accuracy when a large proportion of the data are missing.
  - It has methods for balancing error in class population unbalanced data sets.
  - Fast to build and predict. Fully parallelizable.

- **Disadvantages**
  - Random forests have been observed to overfit for some datasets with noisy classification/regression tasks.
  - For data including categorical variables with different number of levels, random forests are biased in favor of those attributes with more levels.

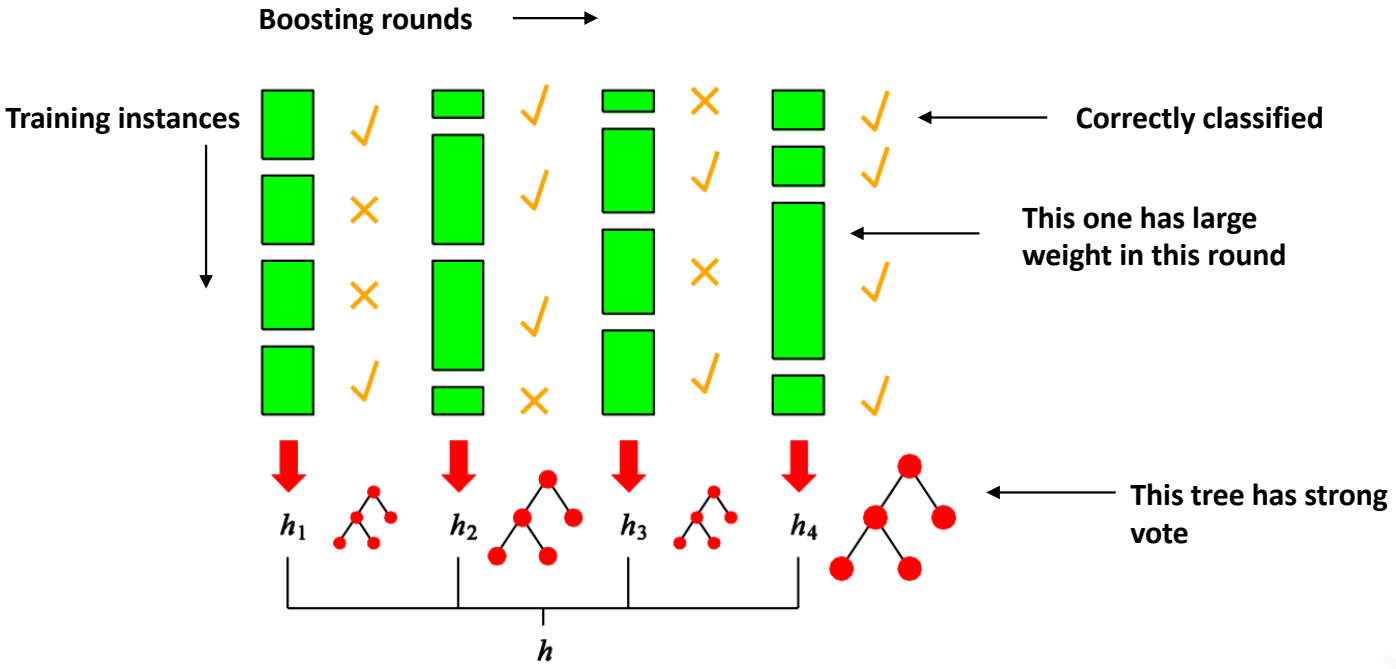
# Boosting

- Training a bunch of individual models sequentially.
- Each model learns from mistakes made by the previous model




https://towardsdatascience.com/basic-ensemble-learning-random-forest-adaboost-gradient-boosting-step-by-step-explained-95d49d1e2725

# Boosting Framework



# AdaBoost

- Adaptive Boosting, formulated by Freund and Schapire (1995)
- The meta-learner adapts based upon the results of the weak classifiers
- Learns from the mistakes by increasing the weight of misclassified data points
- The ensemble model is defined as a weighted sum of weak learners
- Works for both regression and classification problems

# How AdaBoost Works?

Step 0: Initialize the weights of data points.

Step 1: Train a decision tree

Step 2: Calculate the weighted error rate of the decision tree.

Step 3: Calculate this decision tree's weight in the ensemble

$$\alpha_t = \frac{1}{2} ln \left( \frac{1-\epsilon_t}{\epsilon_t} \right)$$

$$\epsilon_t = \frac{\sum_{i=1}^{n} w_i I(y_i \neq h_t(x_i))}{\sum_{i=1}^{n} w_i}$$

# How AdaBoost Works?

Step 4: Update weights of wrongly classified points

Step 5: Repeat Step 1(until the number of trees we set to train is reached)

Step 6: Make the final prediction

For wrongly classified samples, the updated weight will be:
New weight = old weight * exp(weight of the tree(α))

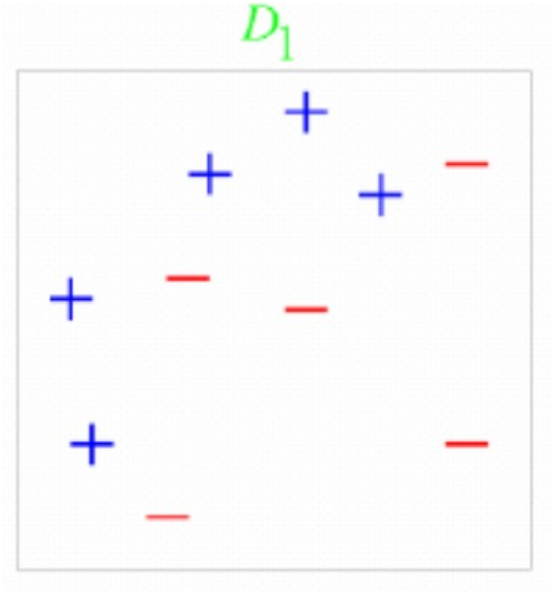
The AdaBoost makes a new prediction by adding up the weight (of each tree) multiply the prediction (of each tree)

$$H(x) = sign\left(\sum_{t=1}^{T} \alpha_t h_t(x)\right)$$

- The tree with higher weight will have more power of influence the final decision

# AdaBoost Example

Training set: 10 points
(represented by plus and minus)
Original Status: Equal Weights for
all training samples



# AdaBoost Example

$$\alpha_t = \frac{1}{2} ln\left(\frac{1-\epsilon_t}{\epsilon_t}\right)$$

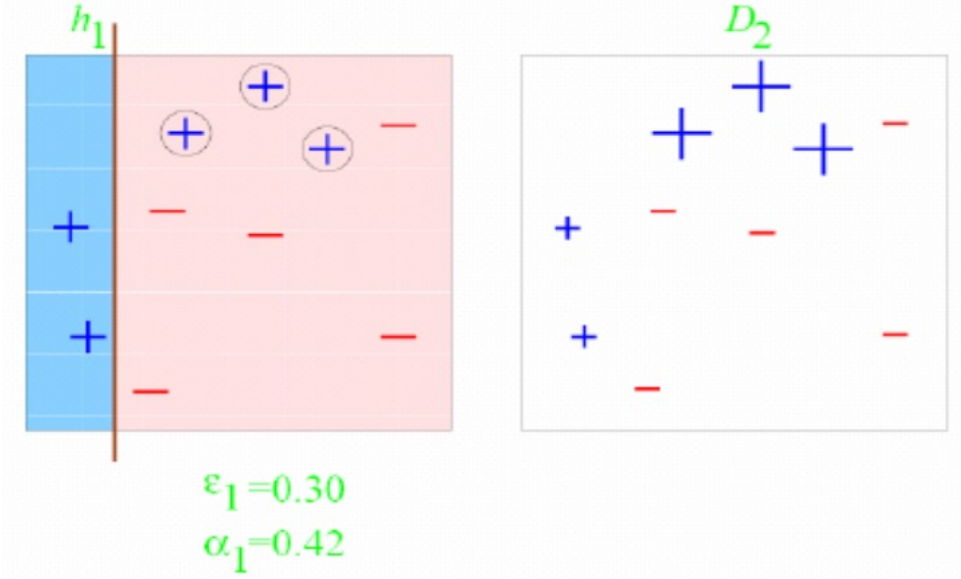
$\epsilon_1 = 0.30$

$\alpha_t = \frac{1}{2} \ln\left(\frac{1-.3}{.3}\right)$

$= \frac{1}{2} \ln \frac{.7}{.3}$

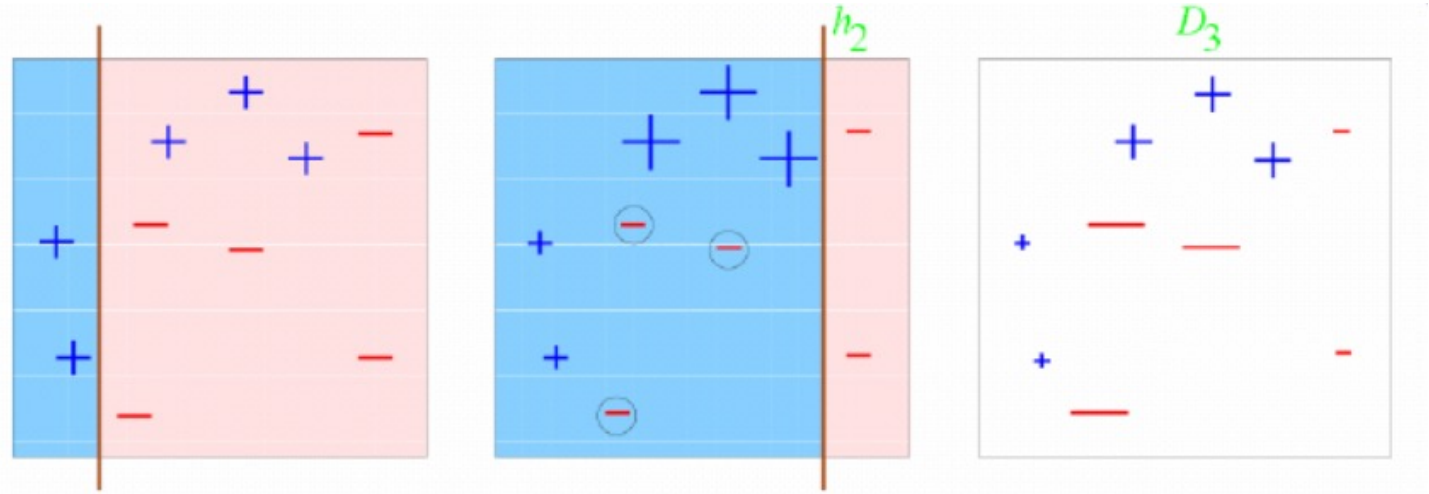
$= \frac{1}{2} \ln(2.33)$

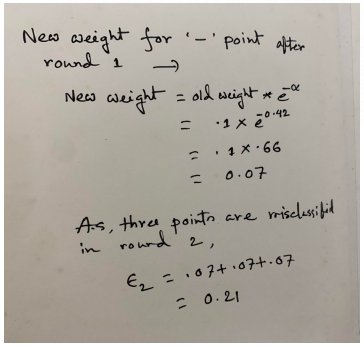
$= 0.42$




Round 1: Three "+" points are not correctly classified;
They are given higher weights.

# AdaBoost Example





New weight for '−' point after round 1 →

New weight = old weight * $e^{-\alpha}$

= .1 × $e^{0.42}$

= .1 × .66

= 0.07

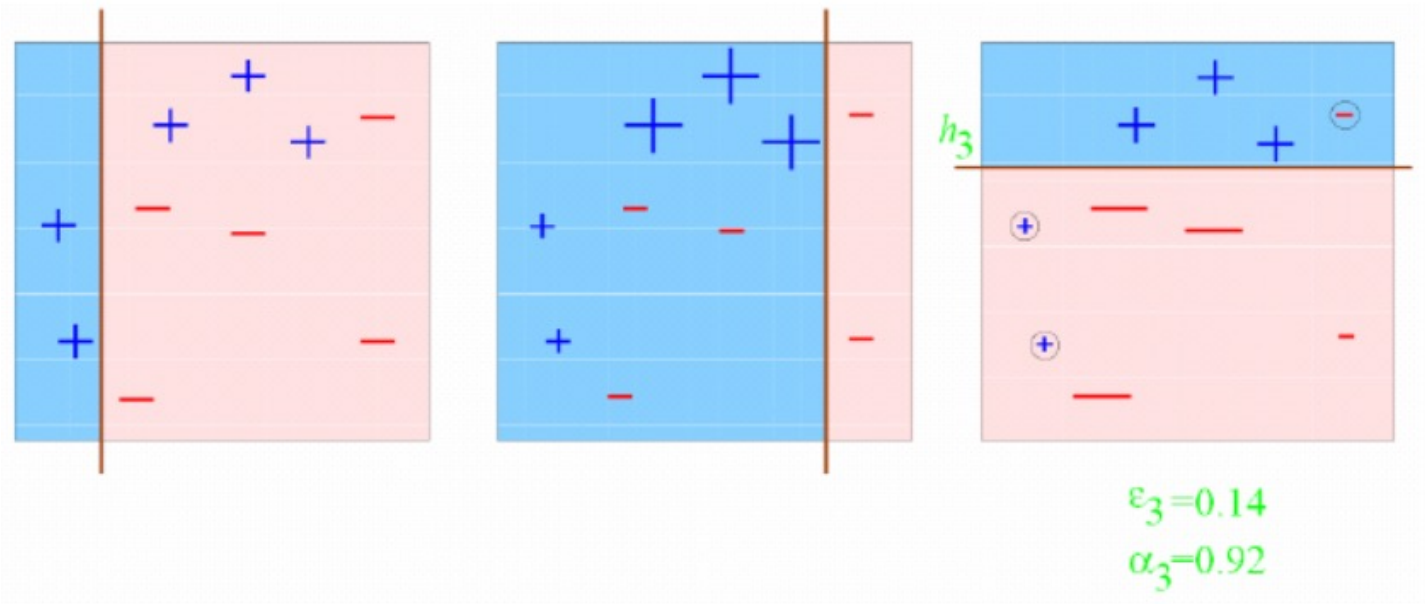
As, three points are misclassified in round 2,

$\epsilon_2$ = .07+.07+.07

= 0.21

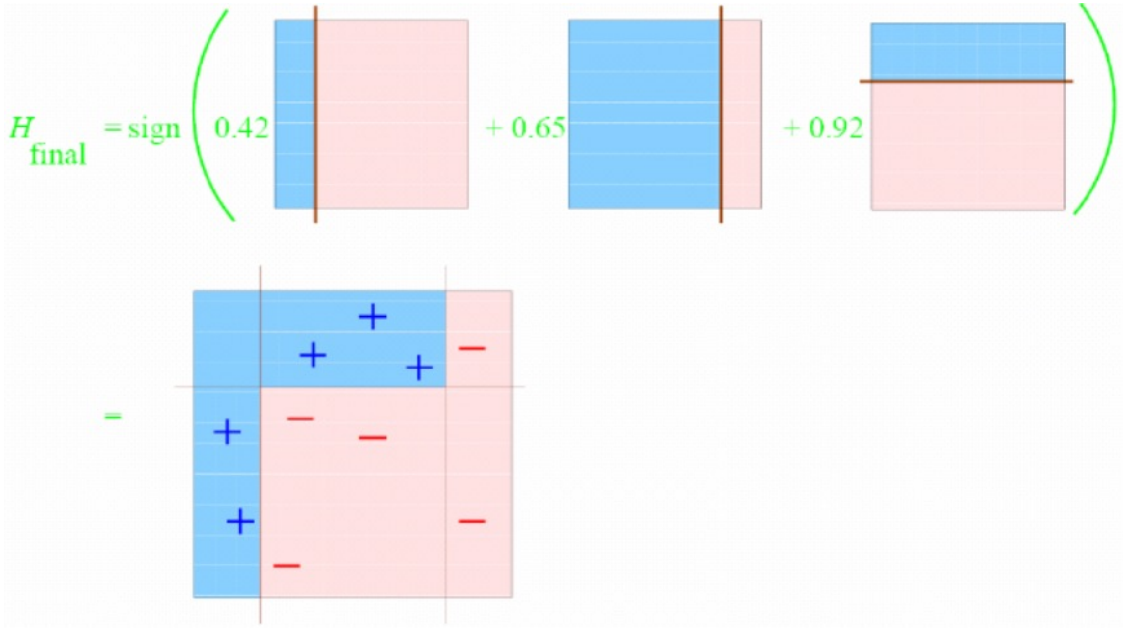
Round 2: Three "-" points are not correctly classified;
They are given higher weights.

# AdaBoost Example




Round 3: One “-” and two “+” points are not correctly classified;
They are given higher weights.

# AdaBoost Example




Final Classifier: integrate the three “weak” classifiers and obtain a final strong classifier.

# What can we boost?

- Weak Learner: produces hypotheses at least as good as random classifier
- Examples:
  - Rules of thumb
  - Decision stumps (decision trees of one node)
  - Perceptrons
  - Naïve Bayes models
- Advantages
  - No need to learn a perfect hypothesis
  - Can boost any weak learning algorithm
  - Boosting is very simple to program
  - Good generalization

# Gradient Boosting

- The idea initially originated by Leo Breiman and subsequently developed by Jerome Harold Friedman
- Works for both regression and classification problems
- Learns from the mistake — residual error directly, rather than update the weights of data points.
- Let's see how gradient boost learns …….

# How Gradient Boosting Works?

Step 1: Train a decision tree

Step 2: Apply the decision tree just trained to predict

Step 3: Calculate the residual of this decision tree, Save residual errors as the new y

Step 4: Repeat Step 1 (until the number of trees we set to train is reached)

Step 5: Make the final prediction

The Gradient Boosting makes a new prediction by simply adding up the predictions (of all trees).

# How Gradient Boosting Works?

- How to calculate residuals?
  - Calculate Log Odds

    -> $\log_e(\frac{p}{1-p})$
  - Calculate probability

    $\sigma = \frac{1}{1+e^{-x}}$
  - Calculate residuals

    residual -> actual value – predicted value

$residual_1$ -> actual value – predicted value (Probability based on class label, CP)

$residual_2$ -> actual value – probability based on $residual_1$($P_{R1i}$)

$P_{R1i} = \sigma(residual_{1i} + \text{learning rate (weight)})$

$$Weight = \frac{\sum residual_i}{\sum Prev\ Pi(1 - Prev\ Pi)}$$

# Gradient Boosting Example (Classification)

- Calculation in the class……(Let's do it together!)

| Age | Income | Credit rate | class | Res1 | Prob1 | Res2 |
|---|---|---|---|---|---|---|
| <=30 | H | F | N | | | |
| <=30 | H | E | N | | | |
| <=30 | L | F | Y | | | |
| [31…40] | H | F | Y | | | |
| [31…40] | L | E | Y | | | |
| [31…40] | M | E | Y | | | |
| >40 | M | F | Y | | | |
| >40 | L | F | Y | | | |
| >40 | L | E | N | | | |

Courtesy: Dr. Niraj Kumar

# * Gradient Boosting Example : (Classification)

— The table includes 9 instances with 3 attributes :
Age, income, credit rate.

Age : <=30, [31 ... 40], >40

Income : H (High), L (Low), M (Medium)

Credit rate : F (Fair), E (Excellent)

Class ~~table~~ labels : Yes (Y), No (N)

Y = 6 , N = 3 $\quad P = 6/9$

Total, n = 9

Compute (Residual 1)

R1

~~Step~~ Log Odds $\rightarrow \log_e\left(\frac{6/9}{1-6/9}\right) = 0.693\ldots = x$

Probability $\rightarrow 6 = \frac{1}{1+e^{-x}} = \frac{1}{1+e^{-(0.693\ldots)}} = 0.7$

Classification Probability

Residual $1_1$ → actual value - predicted value (CP)

$R1_1 = 0 - 0.7 = -0.7$

$R1_2 = 0 - 0.7 = -0.7$

$R1_3 = 1 - 0.7 = 0.3$

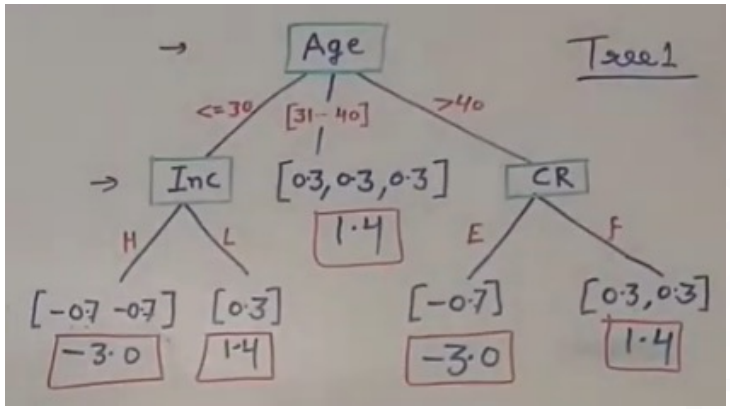
[We consider N=0, Y=1]

:

# Gradient Boosting Example (Classification)

- Calculation in the class……(Let's do it together!)

| Age | Income | Credit rate | class | Res1 | Prob1 | Res2 |
|---|---|---|---|---|---|---|
| <=30 | H | F | N | -0.7 | | |
| <=30 | H | E | N | -0.7 | | |
| <=30 | L | F | Y | 0.3 | | |
| [31…40] | H | F | Y | 0.3 | | |
| [31…40] | L | E | Y | 0.3 | | |
| [31…40] | M | E | Y | 0.3 | | |
| >40 | M | F | Y | 0.3 | | |
| >40 | L | F | Y | 0.3 | | |
| >40 | L | E | N | -0.7 | | |




Courtesy: Dr. Niraj Kumar

## Build Tree 1 →

* Compute Probability based on Residual 1 ($P_{R1i}$) →

$$\text{Weight}_{(\text{leaf node 1})} = \frac{\sum \text{Residuals}_i}{\sum \text{Prev}P_i(1-\text{Prev}P_i)} \quad [\text{for each leaf node}]$$

$$= \frac{-0.7-0.7}{(0.7(1-0.7))+(0.7(1-0.7))}$$

$$= \frac{-1.4}{.21+.21} = -3.33 \quad (\text{for simplicity, we can say } -3.0)$$

$$\text{Weight}_{(\text{leaf node 2})} = \frac{0.3}{0.7(1-0.7)} = 1.4$$

$$\text{Weight}_{(\text{leaf node 3})} = \frac{0.3+0.3+0.3}{.21+.21+.21} = 1.4$$

$$\vdots$$

$$P_{R1_1} \rightarrow \sigma(-0.7 + \underset{\uparrow}{0.2}(-3.0)) \quad (\text{LR})$$

$$= \frac{1}{1+e^{-(-0.7+0.2(-3.0))}} = 0.214 \sim 0.2$$

$$P_{R1_2} = 0.2$$

$$P_{R1_3} \rightarrow \sigma(0.3+0.2(1.4)) = \frac{1}{1+e^{-(0.3+0.2(1.4))}}$$

$$= 0.64 \sim 0.6$$

---

Compute (Residual 2)

$R2$

$$R2_1 = \text{actual value} - P_{R1_1}$$

$$= 0 - 0.2 = -0.2$$

$$R2_2 = 0 - 0.2 = -0.2$$

$$R2_3 = 1 - 0.6 = 0.4$$

## Build Tree 2

→ $P_{R2i}$ →

$$\text{Weight}_{(\text{leaf node 1})} = \frac{\sum \text{residual}_i}{\sum \text{Prev}P_i(1-\text{Prev}P_i)}$$

$$= \frac{-0.2-0.2}{(0.2(1-0.2))+(0.2(1-0.2))}$$

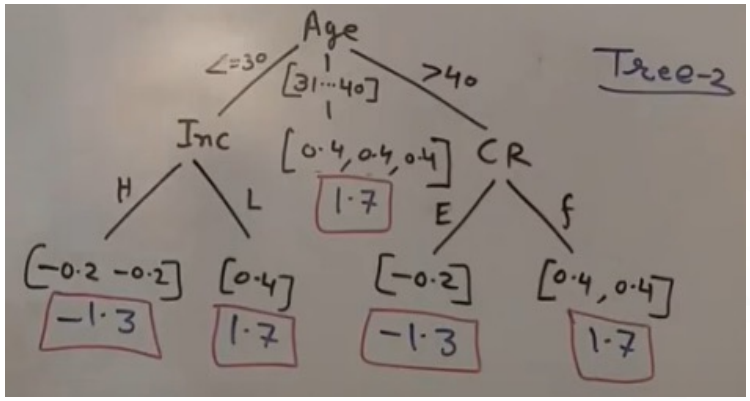
$$= \frac{-0.4}{.16+.16} = -1.25 \sim -1.3$$

$$\text{Weight}_{(\text{leaf node 2})} = \frac{0.4}{0.6(1-0.6)} = 1.66 \sim 1.7$$

# Gradient Boosting Example (Classification)

- Calculation in the class……(Let's do it together!)

| Age | Income | Credit rate | class | Res1 | Prob1 | Res2 |
|---|---|---|---|---|---|---|
| <=30 | H | F | N | -0.7 | 0.2 | -0.2 |
| <=30 | H | E | N | -0.7 | 0.2 | -0.2 |
| <=30 | L | F | Y | 0.3 | 0.6 | 0.4 |
| [31…40] | H | F | Y | 0.3 | 0.6 | 0.4 |
| [31…40] | L | E | Y | 0.3 | 0.6 | 0.4 |
| [31…40] | M | E | Y | 0.3 | 0.6 | 0.4 |
| >40 | M | F | Y | 0.3 | 0.6 | 0.4 |
| >40 | L | F | Y | 0.3 | 0.6 | 0.4 |
| >40 | L | E | N | -0.7 | 0.2 | -0.2 |




Courtesy: Dr. Niraj Kumar

# XGBoost

- Stands for ‘Extreme Gradient Boosting’
- Initially started by Tianqi Chen and maintained by DMLC (Distributed (Deep) Machine Learning Community) group
- An advanced implementation of gradient boosting along with some regularization factors
- Works for both regression and classification problems
- Helps reducing overfitting
- Supports parallel processing

# Other boosting techniques?

- LightGBM
  - LightGBM is a boosting technique and framework developed by Microsoft.
  - Faster training speed and higher efficiency
  - Lower memory usage
  - Compatibility with Large Datasets, etc.
- CatBoost
  - CatBoost is developed and maintained by the Russian search engine Yandex
  - Native handling for categorical features
- ……

# Boosting Applications?

- Any supervised learning task
  - –Collaborative filtering (Netflix challenge)
  - –Body part recognition (Kinect)
  - –Spam filtering
  - –Speech recognition/natural language processing, etc.

# Netflix Challenge

- Problem:

  - predict movie ratings based on database of ratings by previous users

- Launch: 2006
  - –Goal: improve Netflix predictions by 10%
  - –Grand Prize: 1 million $

# Useful Links

- https://towardsdatascience.com/basic-ensemble-learning-random-forest-adaboost-gradient-boosting-step-by-step-explained-95d49d1e2725
- https://www.youtube.com/watch?v=gTUigPt8fVo

Thank you